

Remove complex operands

Für jedes Statement wird geschaut, ob es eine Variablenzuweisung, ein print call oder einfach nur ein arithmetischer Ausdruck ist. Dann wandeln wir die Expressions in diesen Statements mit rco_exp in atomare Expressions um.

Um eine Expression in eine atomare Expression umzuwandeln, matchen wir die Art der Expression und rufen die Funktion rekursiv auf unseren Operanden auf. Diese rekursiven Aufrufe geben uns die atomaren Ausdrücke und eine Liste von Temporaries, dann konstruieren wir einen neuen Temporary und geben diesen zurück.

Wir versuchen auch Temporary Namen wiederzuverwenden, wenn der erste der Operanden als atomarer Ausdruck eine Variable ist. Dies könnte man auch für den zweiten machen, aber wir haben es mehr als proof of concept implementiert. + Der Code an dieser Stelle ist hässlich genug.

Eine weitere Optimierung, die wir an dieser Stelle machen, ist das Entfernen von Statements, die keine side effects haben. Zb. 1+1 (Keine Zuweisung oder Ausgabe).

In diesem Pass nutzen wir unseren eigenen Mechanismus zur Erzeugung von Namen für Temporaries, da wir nicht gesehen haben, dass es im support code schon diese Funktion gibt.

Anzumerken ist, dass es zu Problemen kommt, wenn das Programm Variablen enthält, die wir als Temporaries verwenden. Bsp:

```
tmp_0 = 80
a = 6 +6 +19 --7
print(tmp_0)

-> 31
```

Select instruction

Wir matchen alle Arten von Ausdrücken und übersetzen diese 1 zu 1 in x86_var. Hierbei müssen wir nur aufpassen, dass wir Ausdrücke wie

```
a = a + b
nicht zu
movq b, a
addq a, a
machen.
```

In dem Zug haben wir auch die Optimierung gemacht, dass, falls einer der Operanden auch das Ergebnis ist, wir uns das movq sparen.

Hier war es auch wichtig wirklich alle möglichen Ausdrücke abzudecken, um sie zu übersetzen. Darum werfen wir einen Fehler, wenn wir ein Statement sehen, das wir nicht übersetzen können.

Spätestens hier hätten wir auch ein Problem bekommen, mit Statements wie:
1 + 1 (Keine Zuweisung oder Ausgabe)

Wie würde man das in x86_var übersetzen?

Assign Homes

Jeder Variable wird Platz auf dem Stack zugewiesen, wobei wir mitzählen. Am Ende müssen wir noch dafür sorgen, dass die Stack Größe aufgerundet wird, damit sie durch 16 teilbar ist.

Ansonsten behandeln wir keine besonderen Edge-Cases.

Das bedeutet auch, dass es absolut möglich ist, Variablen zu lesen, die nie im Programm geschrieben wurden. Was zu typos wie diesem führen kann:

```
tmp_0 = 80
a = 6 +6 +19 --7
print(temp_0)

-> 0
```

Patch Instruction

Für zweistellige Instructions, sorgen wir dafür, dass der erste Operand immer ein Register oder Immediat ist.

Für einstellige Instructions, sorgen wir dafür, dass der einzige Operand immer ein Register oder Immediat ist.

Wirklich nicht besonders.

Prelude and Conclusion

Wir fügen Prelude und Conclusion mit der errechneten Stack Größe zum Programm hinzu. Ebenfalls nicht besonderes.